

Cutover al store durable del kernel

Estado de lo hecho, lo que falta y las decisiones pendientes · Issue #5126 · 30 de julio de 2026

Objetivo: que el estado del pipeline deje de vivir sólo en archivos locales y pase a una base remota (DynamoDB), con vuelta atrás ensayada.

Issue: #5126 — Bloque B del cutover (split de #5112 ← #5107 · Ola 9.4)

PR abierto: #5203 · rama `agent/5126-ca-b3-config-kernel` · 2 commits · 8 archivos · 268 tests en verde

Estado del flag: `kernel.durable: false` — el pipeline sigue operando 100% desde archivos. Nada en producción cambió de comportamiento.

1. De qué se trata, en criollo

Hoy el pipeline guarda su estado en archivos JSON dentro del repo (`waves.json` , descriptors, etc.). Eso funciona pero tiene un techo: si la máquina se cae, o si algún día hay más de un pipeline corriendo, no hay una fuente de verdad compartida.

La solución ya estaba **construida y mergeada** hace tiempo (el store DynamoDB, el migrador, el driver, los leases). Estaba **apagada a propósito** con un flag. Esta historia no construye nada nuevo: **enciende lo que ya existe**, y ensaya cómo apagarlo si sale mal.

Para encenderlo hacían falta credenciales AWS que no existían. Eso era el famoso **CA-0**: un prerequisite que sólo podías cumplir vos, porque implica permisos de administrador. Eso es lo que se resolvió en esta sesión.

El camino completo, y dónde estamos parados:

```
CA-0   credenciales + tablas + policy IAM ..... HECHO
CA-B1  postura de seguridad de las tablas ..... HECHO (falta CloudTrail)
CA-B2  permisos mínimos verificados de verdad ..... HECHO
CA-B3  cargar config con el flag TODAVÍA apagado ..... HECHO
CA-B4  migrar datos y verificar que coincidan ..... BLOQUEADO
CA-B5  encender el flag ..... BLOQUEADO
CA-B6  ensayar la vuelta atrás ..... BLOQUEADO
```

2. Qué quedó hecho

2.1 Credenciales acotadas, y un script que las genera solo

Antes, el pipeline usaba el usuario `claude-code` , que tenía **permiso de borrado sobre toda la producción**: las 7 tablas de negocio, las Lambdas, Cognito, los buckets. Un nombre mal escrito en un comando alcanzaba para borrar pedidos de clientes.

Ahora hay un usuario **separado y mínimo** (`intrale-kernel-runtime`) que sólo puede tocar las dos tablas del pipeline.

Vos habías corrido los comandos a mano y la clave secreta se perdió en el camino (AWS no la muestra dos veces). Eso pasó porque el procedimiento era manual. Se reemplazó por **un solo script** que crea todo y **genera y guarda la clave él mismo**: nunca aparece en pantalla, ni en el portapapeles, ni en un historial de terminal.

```
node .pipeline/lib/kernel-aws-bootstrap.js --admin-profile default      # muestra el plan
node .pipeline/lib/kernel-aws-bootstrap.js --admin-profile default --apply  # lo ejecuta
```

Tiene 9 protecciones para no pisar nada en uso. Las tres que más importan:

- Sólo acepta nombres de tabla que empiecen con `intrale-kernel-`. Si te equivocás y ponés `business`, **aborta antes de tocar AWS**.
- Si una tabla ya existe, **no le escribe nada** — ni una fila de prueba.
- El archivo de credenciales se escribe de forma atómica, con backup previo, y el perfil `intrale` que usan el QA remoto y el deploy de Lambda **queda intacto**.

2.2 La política de permisos que te pasé antes estaba mal

Hallazgo: el candado no cerraba

La política que te dicté por Telegram tenía un `Deny` condicionado por `dynamodb:LeadingKeys` con patrones `signature#*` / `audit#*`. Eso **no protege nada**: esa condición evalúa la *clave de partición*, y en el kernel las firmas son *claves de ordenamiento*. La condición nunca se cumple, así que el candado nunca cerraba.

Además cubría 2 de las 7 operaciones que en DynamoDB permiten borrar o pisar un dato.

Ya estaba resuelto en el repo (lo corrigió el Bloque A el 28/07); mi mensaje era de esa misma mañana, anterior al arreglo. La política vigente hoy es la correcta.

2.3 Permisos verificados de verdad, no en el papel

No alcanza con leer la política y decir "se ve bien". Se probó contra AWS real:

Se intentó	Resultado
Borrar una firma de la tabla de no-repudio	RECHAZADO (deny explícito)
Modificar un dato de esa tabla	RECHAZADO
Leer la tabla <code>business</code> (datos de clientes)	RECHAZADO
Crear usuarios o dar permisos en AWS	RECHAZADO
Tomar y liberar un lease (lo que el pipeline necesita hacer)	FUNCIONA

Las pruebas de borrado se hicieron con operaciones reales que rebotaron, no sólo con simulaciones. El dato de prueba que se escribió quedó borrado.

2.4 Cifrado: se cambió la clave por una propia

Las tablas se habían creado con la clave por defecto de AWS. El runbook del proyecto lo prohíbe explícitamente, por tres motivos concretos:

- Una clave de AWS **no tiene política**: no hay dónde declarar quién puede descifrar.
- **No deja rastro auditable** de quién la usó.

- **No se puede deshabilitar:** si perdés control del acceso, no tenés ninguna palanca. Con una clave propia la apagás y todo el contenido queda ilegible en un solo movimiento.

Esto había que resolverlo **antes** de la primera escritura, no después: las firmas y el registro de auditoría son *imborrables por diseño*. Lo que se escriba queda para siempre bajo la clave que hubiera en ese momento.

Se creó la clave propia (`alias/intrale-kernel-store`) con rotación anual, restringida a que **sólo sirva desde DynamoDB**: si alguien roba la credencial del pipeline, no puede usar la clave para descifrar nada por otro lado.

2.5 El error más peligroso de la sesión

Al cambiar la clave, el pipeline perdió acceso a sus propias tablas

La política de permisos termina con una regla que dice "*prohibido todo lo que no sean estas dos tablas*". Con la clave vieja eso era inofensivo. Con la clave nueva dejó de serlo: cada vez que se lee un dato, DynamoDB necesita descifrarlo, y ese permiso se evalúa contra **la clave** — que no es ninguna de las dos tablas. Entonces caía en el "prohibido todo lo demás".

Por qué es peligroso: el comando que aplica la política *sale bien*. El error aparece recién cuando algo intenta leer un dato. Nada avisa en el medio.

No hubo impacto porque el flag está apagado y nadie usa esas tablas todavía. Si se hubiera descubierto con el flag encendido, el pipeline se quedaba sin estado.

Arreglado en dos lugares, no uno: en la política vigente, y en el generador de políticas. Esto último es lo importante — sin eso, la próxima vez que alguien regenerara la política, el problema volvía con un comando de apariencia inocua. Quedó cubierto con tests para que no reaparezca.

2.6 Backups del estado exacto

Antes de tocar cualquier cosa se respaldó el estado del momento, por duplicado:

Qué	Dónde	Contenido
Estado de coordinación	<code>.pipeline/backup/2026-07-30T00-24-30-631Z/</code>	7 olas, 2 bloqueados, 3 bloqueados por infra, 3 de salud — con checksum por archivo
Descriptores	<code>.pipeline/backup/2026-07-30T00-24-44-417Z/descriptors/</code>	<code>intrale-platform.json</code> , 11 registros
Copia cruda extra	fuera del repo	los 5 archivos, verificados byte a byte

La copia cruda se hizo **antes** de usar el mecanismo de backup, a propósito: así el respaldo no depende de que ese mecanismo funcione. Y se ensayó la restauración completa: el archivo volvió con el checksum original intacto.

3. Por qué no se puede seguir todavía

Acá está el nudo, y es lo más importante de este reporte.

3.1 No hay nada que migrar bloquea CA-B4

El criterio pedía "migrar los datos y verificar que coincidan entidad por entidad". Al ir a hacerlo, apareció esto: **no hay datos que migrar**.

El único producto registrado en el pipeline es el propio monorepo, `intrale-platform`. Pero ese identificador está en una **lista de reservados** (decisión de #4853): el monorepo raíz no puede tratarse como un producto más. Se verificó ejecutándolo:

```
registrar el descriptor real como producto
→ RECHAZADO: "id de producto/proyecto reservado"
catálogo resultante
→ vacío (0 productos)
```

Por qué importa: si igual se corriera la verificación de paridad, daría **verde** — porque estaría comparando dos conjuntos vacíos. Ese es exactamente el "falso verde" que el criterio quería evitar. Daría la sensación de que el cutover funcionó cuando en realidad no se probó nada.

Esto también afecta al **CA-B5**: la evidencia de que el pipeline lee de la base no puede ser "listó los productos", porque no va a haber ninguno. Hay que elegir otra prueba.

3.2 El catálogo se guardaría en un lugar y se leería de otro bloquea CA-B5

Al rastrear cómo se poblaría la base aparecieron dos defectos encadenados:

1. **Falta pasar un dato obligatorio.** La función que registra un producto exige saber a qué proyecto pertenece, y el flujo real de alta no se lo pasa. Con el flag encendido, un alta muere con "dato ausente".
2. **Escribe en una partición y lee de otra.** El catálogo se guarda bajo el identificador del producto, pero el arranque del pipeline lo busca bajo un identificador distinto (`kernel-control-plane`). Un catálogo cargado desde un producto es **invisible** para el arranque.

Ninguno se nota hoy porque el punto 3.1 impide llegar hasta ahí. Los dos aparecen al registrar el primer producto real. **Esto es cambio de código**, sobre una parte que ya se había cerrado y mergeado.

4. Lo que necesita tu decisión

Decisión 1 — cómo arreglar la partición del catálogo bloqueante

La opción que parece correcta: que al registrar un producto, el *producto* y el *catálogo* se guarden bajo `kernel-control-plane` (donde el arranque los busca), y sólo el *descriptor* quede en la partición del producto.

Es cambio de código sobre el Bloque A ya mergeado, así que corresponde issue propio y su propio ciclo de QA. **Sin esto, el CA-B5 no puede dar verde honestamente.**

Decisión 2 — reformular los criterios CA-B4 y CA-B5 bloqueante

Tal como están redactados piden una verificación que hoy daría falso verde. Hay que reescribirlos para que digan explícitamente que el conjunto a migrar es vacío y por qué, y para que la evidencia del

encendido sea algo distinto de "listar productos".

Esto es trabajo de definición (PO), no de código.

Decisión 3 — CloudTrail costo

No hay ningún registro de auditoría configurado en la región. El runbook lo pide para poder demostrar quién descifró qué durante la ventana de cutover. Necesita un bucket S3 y tiene costo mensual. Es el único control del cifrado que quedó incompleto.

Decisión 4 — las credenciales de administrador son de la cuenta raíz seguridad

El único perfil con permisos de administración en tu máquina es el perfil `default`, que son credenciales de la cuenta **root** de AWS. AWS recomienda explícitamente no tener claves de acceso de root. Conviene reemplazarlo por un usuario administrador dedicado.

No es urgente ni bloquea nada, pero es la deuda de seguridad más grande que queda abierta.

5. Cómo retomar desde cero

Recursos que ya existen en AWS (región `us-east-2`)

Recurso	Nombre	Estado
Usuario del pipeline	<code>intrale-kernel-runtime</code>	activo, permisos mínimos verificados
Tabla de no-repudio	<code>intrale-kernel-state</code>	activa, protegida contra borrado, PITR y clave propia
Tabla de coordinación	<code>intrale-kernel-coordination</code>	activa, protegida contra borrado, clave propia
Clave de cifrado	<code>alias/intrale-kernel-store</code>	propia, rotación anual, sólo usable desde DynamoDB
Permisos de datos	<code>IntraleKernelStore</code>	vigente, con la clave exceptuada del bloqueo
Permisos de cifrado	<code>IntraleKernelKms</code>	política separada, aditiva
Registro de auditoría	—	no existe (decisión 3)

Dónde están las credenciales

En `~/.claude/secrets/credentials.json` bajo la clave `aws`, y como perfil `kernel-runtime` en `~/.aws/credentials`. Fuera del repo, en los dos casos. El perfil `intrale` de siempre quedó intacto.

Comandos útiles

```
# ver el estado de todo, sin modificar nada
node .pipeline/lib/kernel-aws-bootstrap.js --admin-profile default
node .pipeline/lib/kernel-cmk-provision.js --admin-profile default
```

```
# rotar la clave del pipeline si hiciera falta
node .pipeline/lib/kernel-aws-bootstrap.js --admin-profile default --rotate-key --apply

# los tests de todo lo tocado
node --test .pipeline/lib/__tests__/kernel-*.test.js .pipeline/lib/__tests__/credentials*.test.js
```

Dónde está escrito el detalle técnico

- **PR #5203** — el código, con el razonamiento en los mensajes de commit.
- **Issue #5126** — tres comentarios con la evidencia completa: el cumplimiento del prerrequisito, los hallazgos del CA-B4, y la clave de cifrado con el defecto que apareció.
- [docs/pipeline/runbook-cutover-durable.md](#) — el procedimiento de encendido y vuelta atrás. Se lee *antes* de tocar nada.

6. Resumen en una tabla

Criterio	Estado	Qué falta
CA-0 — credenciales, tablas, permisos	HECHO	tu confirmación formal por comentario en el issue
CA-B1 — postura de seguridad	HECHO	sólo CloudTrail (decisión 3)
CA-B2 — permisos mínimos	HECHO	—
CA-B3 — config cargada, flag apagado	HECHO	—
CA-B4 — migrar y verificar	BLOQUEADO	decisiones 1 y 2
CA-B5 — encender el flag	BLOQUEADO	depende de CA-B4
CA-B6/B7 — ensayar la vuelta atrás	BLOQUEADO	depende de CA-B5

Lo esencial, si te quedás con una sola cosa

La parte de infraestructura está **terminada y verificada**: credenciales mínimas, tablas protegidas, clave de cifrado propia y permisos probados contra AWS real. El flag sigue apagado, así que nada en producción cambió.

El cutover **no puede avanzar** por un motivo que no era visible al planificarlo: el único producto registrado es el monorepo raíz, y está reservado por diseño. Sin productos, la verificación de paridad daría falso verde y el encendido no probaría nada. Eso pide un fix de código y una reescritura de dos criterios — no es algo que se resuelva empujando el cutover.